

Progetto DAPT W8D4

Giovanni M. Martino

September 21, 2026

DAPT_{0326IT}

ToysGroup distribuisce giocattoli in diverse aree geografiche del mondo. I prodotti sono classificati in categorie, i mercati in regione di vendita. Ogni vendita collega un prodotto a una regione

Lo strumento di lavoro che abbiamo scelto per svolgere il task assegnato è SQL, ovvero, un linguaggio di programmazione standard usato per gestire, organizzare e interrogare i database relazionali. Questi ultimi sono un archivio che salva i dati dentro tabelle formate da righe e colonne.

In ciò che segue, quindi, lavoreremo con SQL.

Lavorando in SQL, di conseguenza, dobbiamo scegliere un metodo di presentazione consono che qui, in modo preliminare, dichiariamo.

Al fine di favorire la comprensione di quanto svolto, dei risultati ottenuti e, soprattutto, del quadro complessivo del lavoro, abbiamo scelto di non soffermarci in modo dettagliato sulla spiegazione della sintassi di SQL, per un motivo: abbiamo scelto il metodo di spiegare *cosa* andremo a utilizzare, i nomi delle funzioni, i costrutti, le clausole per spiegare *cosa* servono a calcolare.

Introdurre concetti, progressivamente, per accompagnare l'analisi che abbiamo svolto ci è sembrato il metodo più consono per una presentazione diretta, come in questo caso, a un pubblico non specialista. L'obiettivo, quindi, è accompagnarci insieme in queste slides.

ToysGroup, lo scenario che abbiamo introdotto e su cui lavoreremo, è strutturato in tre aree: *Prodotti*, *Regioni*, *Sales* che, per completezza, qui riportiamo, così da rendere chiaro il metodo di lavoro che implementeremo in SQL.

- Product. Un *prodotto* può essere venduto tante volte o nessuna. Contiene anche le *categorie* di appartenenza (Bikes-100, Bikes-200 appartengono alla categoria Bikes).
- Region. Una *regione* può avere molte transazioni o nessuna. Contiene anche lo stato di appartenenza (Germany, France, appartengono alla regione WestEurope).
- Sales. Ogni transazione è riferita a un solo prodotto. Ogni transazione è riferita a una sola regione (Fatti di vendita: una riga per transazione).

La prima consegna che ci è stata assegnata è quella di individuare le *entità* e le relazioni dello scenario ToysGroup in uno schema Entità/Relazioni.

Che cosa sono, però?

Uno *Schema Entità/Relazioni* (chiamato anche ERD, *Entity Relationship Diagram*) è la rappresentazione grafica del modello concettuale dai dati; proposto da Peter Chen nel 1976, la sua notazione originale, che noi useremo, è ancora lo standard odierno di rappresentazione.

Gli ingredienti dello *Schema E/R* sono, principalmente, tre.

Le *Entità* sono i sostantivi del dominio, ovvero, qualcosa di definibile come una persona, un evento, un concetto che possiamo memorizzare in una tabella: le entità le rappresenteremo in rettangoli.

Le *Relazioni* sono il modo in cui le entità sono associate tra loro all'interno dello *schema*: nella rappresentazione grafica sono delle linee che collegano le entità tra loro.

Gli *Attributi* sono le caratteristiche delle *Entità* e nello *schema* vengono rappresentati con degli ovali accanto all'entità corrispondente. Quelli *Chiave* sono quelli che definiscono in modo univoco una entità come, ad esempio, un *Identificativo* che nello schema sottolineeremo.

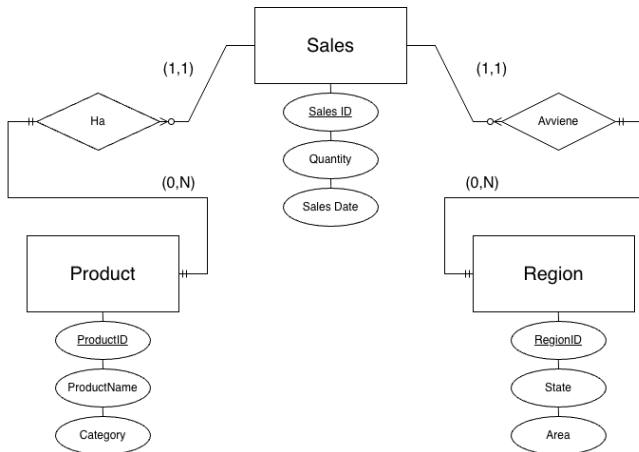
Prima di mostrare lo schema, tuttavia, è necessario introdurre un ulteriore concetto, ovvero, quello di *cardinalità*: quest'ultima ci dice come funzionano le relazioni, ovvero, il numero di istanze che un'entità intrattiene con un'altra entità:

- Uno a Uno (1:1). Una riga della prima tabella corrisponde a una riga dell'altra tabella: dipendente-badge aziendale.
- Uno a Molti (1:N). Una riga della prima, molte righe della seconda: cliente e numero di ordini.
- Molti a Molti (N:M). Molte righe di una corrispondono a molte righe dell'altra: molti ordini e molti prodotti.

Nel seguente *Schema E/R* mostreremo, in ordine:

- Per ciascuna Entità (*Product*, *Region*, *Sales*) attributo chiave e attributi descrittivi.
- Descriveremo la cardinalità delle relazioni *Product-Sales* e *Region-Sales*.
- Vedremo le gerarchie: *Product* include *Category*, *Region* include *States*.

Schema E/R, *progettazione concettuale*



Chiavi primarie, esterne e tipi di dato

La seconda parte della prima Task riguarda il passaggio da progettazione concettuale a *progettazione logica*: la consegna richiesta, quindi, è di tradurre lo schema precedente in una tabella relazionale con colonne e chiavi esterne. Anche qui, propedeuticamente, occorre chiarire i termini e come svolgeremo il lavoro.

Chiave Primaria (PK). È la colonna o l'insieme di colonne che identificano in modo univoco ogni riga della tabella.

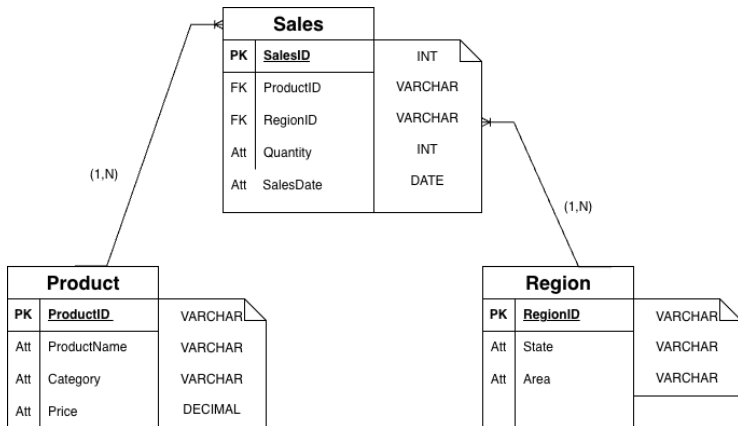
Chiave Esterna (FK). È una colonna che contiene i valori della chiave primaria di un'altra tabella: è il collegamento tra due tabelle.

Tipo di Dato. Ovvero, il tipo di dato che assegneremo a chiavi e attributi in SQL, cioè, INT (per i numeri interi), VARCHAR per le stringhe di lunghezza variabile, DECIMAL per i prezzi, DATE per le date (appuntamento).

Nel seguente *Schema logico* mostreremo, in ordine:

- L'elenco di ciascuna tabella (*Product*, *Region*, *Sales*) che implementeremo, non solo le tabelle.
- Dichiareremo la Chiave Esterna (FK) di *Sales* verso *Product* e verso *Region*, con cardinalità N:1.
- Verificheremo che la struttura minimizza la ridondanza, ovvero, che ogni chiave esterna referencia una chiave primaria esistente.

Progettazione logica



Task 1b, *Progettazione logica*

- Una sola PK per tabella: Product verso ProductID, Region verso RegionID, Sales verso SalesID;
- Ogni FK referencia una PK esistente: Sales.ProductID referencia Product.ProductID, Sales.RegionID referencia Region.RegionID;
- Cardinalità N:1 rispettata: molte righe di Sales possono avere lo stesso ProductID (lato N in Sales, lato 1 in Product); molte righe di Sales possono avere lo stesso RegionID (lato N in Sales, lato 1 in Region);
- Minimizzazione della ridondanza: nessun dato di Product o Region è duplicato dentro Sales: la tabella dei fatti contiene solo gli ID (le FK), non i nomi o le categorie.

Dalla progettazione all'implementazione

La primissima parte del nostro lavoro si è concentrata sulla progettazione prima concettuale, poi logica, dello scenario di ToysGroup. Il prossimo passo è implementare il nostro lavoro nel linguaggio che abbiamo presentato in apertura di presentazione, ovvero, SQL, che useremo tramite Workbench, un'interfaccia grafica per progettare, sviluppare e amministrare database in SQL: in altre parole è un programma che abbiamo sul nostro computer per gestire database locali (come in questo caso) o remoti.

La sintassi che useremo è chiamata DDL (Data Definition Language) e ci servirà per definire, modificare, creare la struttura del nostro database attraverso tabelle, indici o schemi.

Prima di cominciare a mostrare l'implementazione del nostro lavoro è bene introdurre brevemente la sintassi che useremo.

In primo luogo ci occuperemo di creare il database, con un apposito comando, CREATE; successivamente, tramite USE, cominceremo a renderlo operativo. Tuttavia, per creare le tabelle a partire dalle entità già introdotte, *Product Sales* e *Region* utilizzeremo il seguente schema:

- CREATE TABLE per creare la tabella, con relativo nome;
- il nome della colonna (*ProductID*, ad esempio) con relativo tipo di dato: INT per numeri, VARCHAR per le stringhe (con lunghezza massima inserita), DECIMAL per numeri decimali, DATE per le date;
- Dichiareremo se sono PK o FK (chiavi primarie o esterne) e con REFERENCES il collegamento tra colonne.

Task 2. DDL e creazione tabelle

In accordo la sintassi introdotta in precedenza, abbiamo implementato lo scenario ToysGroup come segue:

La prima tabella introdotta è stata *Product* dove: *Product ID*, *ProductName* e *Category* sono stati tipizzati come VARCHAR e *Price* come DECIMAL. La PK è, ovviamente, *Product ID*.

La seconda tabella introdotta è stata *Region* dove: *RegionID*, *State* e *Area* sono stati tipizzati come VARCHAR. La PK è *RegionID*.

L'ultima tabella introdotta è stata *Sales* dove: *ProductID* e *RegionID* sono stati tipizzati come VARCHAR, *SalesID* e *Quantity* come INT, *SalesDate* come DATE. La PK è *SalesID*, le FK *ProductID* e *RegionID* sono state referenziate a *Product* e *Region*.

Sono stati rispettati i vincoli imposti, ovvero, NOT NULL per gli attributi obbligatori, (cioè nessun valore nullo possibile in quelle colonne), tipo numerico in base al dato, stesso tipo tra PK e FK.

Una volta entrati nell'ambiente di lavoro di SQL, prima di procedere all'interrogazione del database (che stiamo sviluppando), occorre fare ancora un ultimo lavoro preliminare, ovvero, *popolare* le tabelle che abbiamo creato con dati a scelta. La richiesta che ci è stata fatta è di inserire pochi record per tabella con, tuttavia, alcuni vincoli da seguire.

- Inserire nella tabella *Product* almeno 4 prodotti su 2 categorie diverse;
- In *Region* di inserire almeno 3 stati distribuiti su almeno 2 regioni di vendita differenti;
- In *Sales* almeno 10 transazioni distribuite su più anni, in modo da confrontare periodi diversi.

L'inserimento in SQL avviene tramite il comando INSERT dove, per ognuna delle tre entità, popoliamo le righe corrispondenti con dati (es. ProductID, ProductName, Category).

Task 3. *Popolamento dei dati*

Per generare dei dati utili a popolare le nostre tabelle, abbiamo utilizzato uno dei moderni chatbot di AI per generare dati in modo rapido ed efficiente.

- In particolare, abbiamo inserito in *Product* 25 prodotti distribuiti in 4 categorie differenti.
- In *Region* 10 stati in 3 aree geografiche differenti.
- In *Sales* 25 prodotti distribuiti su anni diversi.

Usando il comando INSERT INTO, abbiamo selezionato, per esempio, la tabella *Product* e l'abbiamo popolata con i relativi attributi (Bikes-100, Mountain Bike Explorer, Bikes, 350.00).

La prima fase del nostro lavoro, quindi, può dirsi conclusa. Abbiamo prima concettualizzato lo scenario che ci è stato proposto con uno schema E/R che abbiamo successivamente tradotto in uno schema logico per tabelle; abbiamo cominciato a lavorare in SQL creando il database ToysGroup, generando le tre tabelle di riferimento (*Product*, *Region* e *Sales*), popolando poi il database come mostrato nella precedente slide.

La seconda fase, ora, si svilupperà interamente all'interno di SQL e l'obiettivo sarà quello di interrogare il database ToysGroup rispondendo, quindi, ai quesiti che ci saranno posti in merito ai dati. Sarà un lavoro più complesso da un lato, ma anche più divertente dall'altro, dove entreremo più nel dettaglio della sintassi di SQL. Tuttavia, ogni passaggio sarà spiegato in modo propedeutico e, possibilmente, la chiarezza espositiva non dovrebbe risentirne: qualcosa della sintassi non sarà chiara ma la spiegazione di come ci siamo arrivati e del risultato ottenuto, sì.

Il concetto di Join e di booleano

Per cominciare, introduciamo il concetto principale che introdurremo nella prossima Task, ovvero, la *JOIN*.

In SQL la JOIN è un'operazione che permette di unire e combinare i dati provenienti da due o più tabelle in un unico risultato, utilizzando relazioni logiche come le chiavi esterne che sono quelle, come visto in precedenza, che collegano le tabelle tra loro. Esistono vari tipi di JOIN, quella di nostro interesse però è la INNER JOIN, ovvero, quell'operazione che restituisce come risultato solo le righe che hanno una corrispondenza in entrambe le tabelle.

Il secondo concetto che introduciamo è quello di condizione booleana, ovvero, quel tipo di dato che può assumere soltanto due valori, vero o falso. Entrambi i concetti saranno fondamentali per la prossima Task.

Rispetto a ciò che abbiamo introdotto, le prime richieste di interrogazione del database ToysGroup riguardano la verifica delle chiavi primarie (le PK precedentemente introdotte) e, tramite il costrutto che abbiamo appena introdotto, INNER JOIN, costruire l'elenco delle transazioni.

- Scriveremo una Query (ovvero una richiesta di informazioni al database) per verificare che le PK sono uniche.
- Useremo INNER JOIN per avere, in ordine, codice del prodotto, categoria, stato, regione di vendita per ogni transazione.
- Applicheremo la condizione booleana: True se sono passati più di 180 dalla vendita, False altrimenti.

Task 4a, *Integrità e Join*

Per il primo punto del Task, abbiamo utilizzato SELECT con COUNT, una funzione di aggregazione che serve a contare il numero di righe o di valori presenti in una tabella, quindi con COUNT DISTINCT abbiamo contato il numero di valori unici presenti *Product*, *Region* e *Sales*.

Per il secondo, il SELECT è stato fatto da Product ID, Category, StateName, AreaName e SalesDate e con l'INNER JOIN, a partire dalla tabella *Sales* abbiamo cercato i valori comuni in *Product* e *Region*.

Infine, per la condizione booleana, abbiamo riscritto la stessa query aggiungendo, la clausola DATEDIFF, ovvero, un modo per calcolare la differenza tra due date, tra quella odierna e i 180 giorni.

Il concetto di aggregazione e di raggruppamento

I prossimi concetti da introdurre preliminarmente, per procedere con la nostra interrogazione del database ToysGroup, sono due importanti clausole di SQL, ovvero, *GroupBy* e *Having*.

GROUP BY serve a *raggruppare* le righe che hanno gli stessi valori in una o più colonne, in modo da fare calcoli di riepilogo sui singoli gruppi. La useremo, quasi sempre, assieme a *funzioni di aggregazione*, come COUNT, già introdotto in precedenza, per contare gli elementi, SUM per sommare valori e AVG per calcolare la media.

HAVING, invece, serve a *filtrare* i risultati dopo che i dati sono stati raggruppati tramite GROUP BY. HAVING, di conseguenza, agisce dopo il raggruppamento, permettendo di applicare condizioni sulle funzioni aggregate elencate sopra.

Rispetto a ciò che abbiamo introdotto nella precedente slide, la richiesta di analisi riguarda la verifica del fatturato aggregato su diverse chiavi tramite, ovviamente, le due clausole introdotte.

- Calcoleremo il fatturato totale per prodotto, per anno, raggruppato per *Product ID* e *Sales Date*.
- Ancora, calcoleremo il fatturato totale per anno e per stato, ordinato per data e per fatturato decrescente.
- Verificheremo la quantità di prodotti più richiesti dal mercato, misurandoli come quantità totale venduta.

Vediamo ora il fatturato e come abbiamo svolto il calcolo.

Task 4b, *Aggregazioni e raggruppamenti*

Per il calcolo del fatturato totale per prodotto, abbiamo prima collegato *Sales* e *Product* tramite *Product ID*; realizzata la somma totale come il rapporto tra prezzo e quantità, raggruppato le righe con la combinazione prodotto, infine, abbiamo filtrato quelle con il fatturato maggiore. Come risultato abbiamo, quindi, una riga per ogni coppia (prodotto, anno) in cui quel prodotto è stato venduto, con il fatturato totale di quell'anno. Ad esempio, nel nostro scenario, nel 2022 il totale di Bikes-100 è stato 4200.00 e nel 2023, Cars-300 440.00 .

Per la seconda richiesta, abbiamo replicato la query precedente aggiungendo solo *Region* così da poter raggruppare per stato e anno ordinando, infine, il fatturato. Il risultato è così una tabella con Stato, anno e il complessivo, *Sales Amount*.

Infine, per la categoria più richiesta abbiamo fatto una semplice somma della quantità sulle 4 categorie. Nel risulta, nel nostro scenario, *Dolls* la più venduta con 104.

Un passo avanti: CTE e Subquery

La nostra analisi del Database ToysGroup sta procedendo per gradi: per ogni task richiesta aggiungiamo un livello di complessità nella scrittura della query in SQL che, tuttavia, introduciamo per rendere comprensibile il lavoro che abbiamo svolto. Passiamo ora a query più strutturate, ovvero, che richiedono più condizioni all'interno tramite, appunto, altre query all'interno della principale.

Una subquery è una query SQL inserita all'interno di un'altra istruzione: la usiamo per calcoli o per recuperare dati da usare nella query principale. La scriviamo tra parentesi tonde e la eseguiamo prima della query principale: ci dà, sempre, un valore singolo, un elenco o una tabella.

Una CTE (*Common Table Expression* o espressione di tabella comune) invece, è una tabella virtuale o temporanea, definita con un nome, incorporata all'interno di una query introdotta. È anticipata dalla clausola **WITH** e poi da **AS** con il nome scelto. Semplifica le query complesse rendendo il codice più pulito e facile da leggere.

La prossima richiesta riguarda l'analisi quantitativa che impiega però i costrutti appena introdotti: l'analisi dei prodotti venduti con quantità totale superiore alla media nell'ultimo anno censito.

- Tramite una subquery, calcoleremo la quantità media di un prodotto nell'ultimo anno censito.
- Useremo la subquery costruita sopra in una condizione WHERE, per filtrare i prodotti sopra media.
- Riscriveremo, infine, la stessa subquery ma con una CTE che isola il calcolo della media.

Nota. La clausola WHERE, che ancora non avevamo introdotto nel contesto della nostra presentazione, serve a filtrare le righe (o, meglio, i record) di una tabella per estrarre solo quelli che soddisfano una determinata condizione.

Task 4c, Subquery e CTE

Per la prima richiesta, la subquery creata è stata generata dal totale delle vendite filtrando con WHERE l'ultimo anno che ci interessa; poi, tramite il raggruppamento per *ProductID* abbiamo generato una tabella che prende i totali calcolati sopra e ne fa la media. Nel nostro scenario, 13.713.

Successivamente, abbiamo ripreso integralmente la subquery di sopra e l'abbiamo inserita in una condizione WHERE: ovvero, prima ci siamo calcolato il totale venduto per prodotto con la subquery, poi l'abbiamo inserita in WHERE che ci ha filtrato solo quei prodotti con la media superiore a quella calcolata sopra. Nel nostro scenario, abbiamo Cars-500 con 21, Dolls-400 con 22 e Dolls-500 con 19.

Infine, abbiamo utilizzato il metodo della CTE: abbiamo denominato l'intera tabella come *TotaliPerProdotto* richiamandola poi nella query principale ottenendo, ovviamente, il medesimo risultato della richiesta precedente.

Il prossimo step della nostra analisi si concentrerà su una analisi delle transazioni per giungere a un dettagliato totale progressivo. Per fare ciò, dobbiamo introdurre il concetto di *Window Function*.

Le WF sono funzioni che permettono di eseguire calcoli su un insieme di righe correlate alla riga corrente: si differenziano da GROUP BY perché non raggruppano le righe in un unico risultato, ma mantengono il dettaglio di ogni riga.

Sintatticamente la introduciamo sempre con la clausola OVER che utilizza due costrutti: PARTITION BY, che va a dividere le righe in sotto-gruppi (senza di questo la tabella verrebbe considerata come un unico grande gruppo) e ORDER BY che definisce l'ordine in cui le righe vengono elaborate all'interno della funzione.

Di conseguenza, le richieste del prossimo task riguarderanno una dettagliata analisi totale delle transazioni (il nostro result Set) con una classifica (*Rank*) e un totale progressivo, dettagliato per riga.

- All'interno della propria categoria, daremo a ogni prodotto una posizione in classifica per fatturato totale.
- Calcoleremo, per ogni transazione, il totale progressivo del fatturato della regione fino a quella data.
- Confronteremo il fatturato di ogni transazione con quello della transazione precedente della stessa regione.

Il calcolo, più complesso nella scrittura solo in modo apparente, lo vedremo nella prossima slide.

Task 4d, *Windows Function*

Per la prima richiesta. Prima di calcolare il *Rank*, il database esegue la query interna per calcolare i totali di vendita (con il SUM); poi, con l'INNER uniamo le vendite ai prodotti e con GROUP BY li raggruppiamo per avere una riga unica per prodotto. Dopodichè, la WF viene applicata: dividiamo i prodotti in base alla categoria, li ordiniamo per il fatturato e con *Rank* gli assegniamo la posizione in classifica.

La seconda query, invece, ha un funzionamento più lineare: prima abbiamo calcolato la somma del fatturato per ogni transazione poi, su quella, abbiamo applicato la WF in modo da ottenere il progressivo richiesto.

L'ultima, invece, è risultata più complessa. Abbiamo prima impiegato una CTE per organizzare un totale progressivo, unito le righe dei fatturati, corrente e precedente e, infine, la query principale mostra i dati correnti, recupera il fatturato precedente e calcola la differenza. La prima vendita di ogni regione restituisce NULL perché non ha precedente.

Ci avviamo, ora all'ultima parte della nostra analisi. Se fin qui l'interrogazione del database ha riguardato esclusivamente richieste sui prodotti venduti, l'ultima parte di analisi riguarderà, invece, quelli non venduti. Per far ciò, introduciamo gli ultimi concetti di questa nostra presentazione.

Una VIEW, o vista, è una tabella virtuale basata sul risultato di una query SELECT. Non è una normale tabella in quanto non memorizza fisicamente i dati su disco o server. Quando interroghiamo una vista dobbiamo pensare a una query salvata all'interno del database a cui è stato dato un nome, che possiamo utilizzare come se fosse una normale tabella.

La *normalizzazione* e la *denormalizzazione* sono due tecniche di progettazione dei database relazionali: la prima elimina la ridondanza, la seconda ottimizza le performance. (Se voglio il totale di ogni ordine, con la prima calcolo il totale tramite somma dei prezzi, con la seconda aggiungo una tabella totale ordini.)

L'ultima richiesta riguarda i prodotti non venduti, e la creazione di due viste che ci serviranno per il lavoro di reporting.

- Individueremo i prodotti non venduti con un approccio a scelta.
- Faremo il medesimo calcolo con un modo diverso.
- Creeremo una vista, cioè, una tabella temporanea in una versione denormalizzata che avrà codice prodotto, nome prodotto e nome categoria.
- Infine, faremo una vista per area geografica al fine di renderla utile a chi farà analisi delle aree di vendita.

Vediamo, ora, gli ultimi calcoli di questo nostro lavoro.

Task 4e, *Prodotti invenduti e VIEW*

Per l'individuazione dei prodotti invenduti:

Abbiamo preso l'elenco completo di tutti i prodotti e lo abbiamo unito alla tabella delle vendite. Poi, con un LEFT JOIN (che tiene sempre tutte le righe della tabella di sinistra), manteniamo tutti i prodotti nel risultato finale. Infine, dopo aver collegato le tabelle *Product* su *Sales*, con la WHERE filtriamo cercando solo i prodotti che non sono stati venduti.

Per l'alternativa abbiamo usato NOT EXISTS: la query principale scorre la tabella dove, per ogni prodotto, viene eseguita la sottoquery con SELECT, se questa trova un risultato in *Sales* lo da, allora NOT EXISTS diventa falso e viene scartata. Se SELECT non trova nulla, invece, il prodotto viene inserito nella tabella finale.

Le due viste, infine, sono di semplici lettura: con CREATE VIEW abbiamo detto al database di memorizza il nome assegnato interrogandola, poi, come fosse una tabella normale con i dati che la popolano.

L'ultima parte del nostro lavoro riguarda una breve analisi della Governance e della Privacy rispetto al nostro utilizzo e condivisione di dati di un Database. In modo preliminare, tuttavia, introduciamo qualche concetto guida.

Il regolamento che disciplina i dati personali nell'Unione Europea è il GDPR (Regolamento UE 2016/679). Ogni trattamento dei dati dovrà rispettare i principi di *liceità*, ovvero, una base giuridica di sostegno; *minimizzazione*, si devono raccogliere solo i dati necessari allo scopo; *limitazione della conservazione*, cioè, la conservazione dei dati solo per il tempo strettamente necessario allo scopo, dopodiché devono essere cancellati. Infine, *diritti dell'interessato*, accesso, rettifica, portabilità e cancellazione dei dati.

Quando abbiamo a che fare con dati sensibili (categorie particolari di dati, art.9 GDPR che se trattati senza garanzie esporrebbero l'interessato a rischi elevati) nel modello di dati le colonne che contengono questa tipologia di dati sono da isolare, vanno documentate e protette con controlli più stringenti delle altre.

In base a questo, allora, vediamo di applicare queste linee guida nell'ultimo task del nostro lavoro.

Ultimo Task, Governance e Privacy applicata

Le quattro strutture che ci sono state presentate violano una delle regole che abbiamo menzionato nelle precedenti slides. Andiamo a vederle.

1. Vista pubblica. La Vista espone *PurchaseCost* (costo di acquisto/produzione) a un pubblico esterno di rivenditori che è un dato commerciale sensibile. È una informazione che non deve mai uscire fuori dall'organizzazione/azienda.
2. Scheda fornitori. I dati di contatto violano il principio di minimizzazione: i dati di contatto dei fornitori (Phone) vengono condivisi con il reparto marketing, che non ha alcuna necessità operativa di quell'informazione.
3. Vista commerciale. La vista espone *PurchaseCost* e *Margin* (margine di guadagno) al reparto commerciale/vendite: sono entrambi dati riservati che non devono essere condivisi con l'esterno.
4. Log di reporting. La tabella registra query e utenti senza alcuna scadenza: in altre parole viola il principio di limitazione della conservazione in quanto conserva log a tempo indeterminato.

Diamo, infine, uno sguardo su quanto abbiamo fatto.

Il lavoro è stato svolto utilizzando il linguaggio di programmazione per database relazionale SQL tramite l'interfaccia Workbench e il database, da noi creato, ToysGroup, con relativo scenario.

È molto probabile che nella risoluzione della task che ci sono state assegnate non abbiamo sempre usato il metodo di risoluzione migliore: anzi, alcune volte è stata tralasciata sicuramente la strada migliore per una probabilmente più complessa e meno intuitiva.

All'interno della nostra presentazione, invece, più che la spiegazione dettagliata della sintassi che abbiamo utilizzato nelle query di SQL, abbiamo scelto il metodo di spiegare *cosa* abbiamo utilizzato per calcolare *cosa*, spiegando dunque i costrutti, le clausole più che il dettaglio della sintassi di SQL. Molte cose, quindi, le abbiamo tralasciate consapevoli, però, della strada che abbiamo scelto di intraprendere.

Speriamo quindi, in conclusione, nella buona riuscita di quanto è stato presentato.

Grazie!